

Lab assignment: NumPy

Applied programming (LIS050)

Robert Östling
robert@ling.su.se

Amanda Kann
amanda.kann@ling.su.se

February 9, 2026

1 Purpose and learning outcomes

In this assignment, you will learn the basics of the NumPy library and parts of the ecosystem of libraries built around it.

To pass this assignment (without distinction), you will need to:

- Implement the multilingual text search specified in Section 3 using NumPy datatypes.
- Review of the code from another student, and demonstrate/explain your code to that student. A checklist should be submitted on Athena.

To pass with distinction, you will additionally need to:

- Implement code to create the multilingual word vectors used in the search algorithm, specified in Section 4

The data and code required for this lab is all located on the server `triton.dsv.su.se` in the directory `/home/dsv/mariask/courses/lis050/lab2`. Please **do not** copy the large data files to your own home directory.

2 NumPy and friends

NumPy is a library for vector operations in Python, and almost has the same status as the built-in libraries that ship with Python. A large number of other libraries build on or integrate with NumPy. Common libraries that build on NumPy include SciPy (various types of numerical computing, can be seen as an extension of NumPy itself), scikit-learn (machine learning), pyTorch (machine learning), and many others.

This lab assumes that you have read up on basic NumPy usage at <https://numpy.org/doc/2.2/user/quickstart.html>, and the fundamentals of vector semantics in Jurafsky and Martin (2024, Sections 5.1–5.4).

3 Multilingual text search

3.1 Introduction

It has been standard in NLP for the last 15 or so years to represent words using vectors. There are many ways to do this, but the most straightforward way, which we will be using here, is to represent each word form by one vector. The dimensionality of these vectors varies, but 300 dimensions (as we will be using) is a common choice.

We are using multilingual word vectors from the ConceptNet Numberbatch project, which provides 300-dimensional word vectors for 79 different languages. These vectors have one important property that we will make use of: the semantic similarity of words a and b can be approximated using the cosine of the angle between their respective vectors v_a and v_b , which can be obtained from:

$$\frac{v_a \cdot v_b}{||v_a|| \cdot ||v_b||} \quad (1)$$

This is true even if a and b are from different languages.

You will implement a system to search through a database of news text. The system should be able to accept a search query in one language, and return hits in another language.

The word vector files from ConceptNet Numberbatch are stored as one file per language, and are located in the following directory:

`/home/dsv/mariask/courses/lis050/lab2/conceptnet-numberbatch.`

The file format is simple: the first line contains two space-separated numbers, the vocabulary size (varies by language) and the dimensionality (in the data above, this is always 300). All following lines contain a word, followed by the vector itself as a list of 300 floats. Between the word and each float is a space character, so you can simply use the `split()` method to separate them.

You can have a look at the data (always a good idea) using a command like this:

```
zcat en.vec.gz | less
```

3.2 Task

Your task is to write two classes in the file `text_search.py`: `WordVectors` and `TextSearch`.

The class `WordVectors` should have the following methods:

- A constructor that takes a filename containing an embeddings file of the format we use (see above).
- `make_sentence_vector(words: list[str]) -> np.ndarray` that computes the mean vector of all the word *types* in the given sentence. Only

words that occur in the vocabulary of the word vectors should be included, and each word should be counted only once even if it occurs multiple times. The word vectors only contain lower-case words, so all words in the input should be lower-cased. The returned array should be a vector of the same dimensionality as the word vectors (in this case, 300, but your code should be general enough to handle other cases as well). If none of the words has a word vector associated with it, raise a `ValueError` exception. Use the datatype `numpy.float32` when creating the NumPy vectors, in order to conserve memory (see the NumPy introduction).

The class `TextSearch` should have the following methods:

- A constructor that takes a list of filenames with word vectors. The name of the file is of the form `de.vec.gz` (for German, with language code `de`). This is important, so that you can keep track of which language each file's word vectors refers to, which allows you to support searches in many different languages. Hint: the `os.path.basename` function in combination with `split(".")` is useful here.
- `index_text(filename: str, language_code: str, min_words: int = 1, max_words: int | None = None) -> None` which takes a filename of a gzip-compressed text with one tokenized sentence per line, reads it and computes and stores sentence vectors for each sentence. The exception is sentences with no words in the vocabulary of the given language's word vectors, or sentences that are shorter or longer than the `min_words` and `max_words` arguments specify. If one or both of the limits is `None`, that particular limit(s) does not apply. Such sentences should be ignored and not indexed. Use the `make_sentence_vector()` method to compute the sentence vector. (Note that `index_text` can be called several times, thereby making it possible to search in several different files. The language of the file content that is to be indexed is specified by `language_code`.)
- `search(query: list[str], language_code: str, n_matches: int = 1) -> list[tuple[float, str, str]]` that returns the `n_matches` (with a default value of 1) sentences most similar to the query, among all of the texts that have been loaded, with the most similar first. Each tuple in the returned list should contain the following three items:
 1. The cosine similarity between the query and the sentence.
 2. The file name of the text file (the same that was passed to `index_text()`).
 3. The sentence itself as a string, exactly as found in the input text file.

The `language_code` parameter should match one of the language codes in the file names given to the constructor. Note that this method should not read any files, but rely on pre-computed data stored in the `WordVector` object.

You should now be able to call the test program `/home/dsv/mariask/courses/lis050/lab2/test_text_search.py`. You call this program by using a configuration file written in json. This configuration file specifies two english text collections that are to be indexed for search, and two word vector models that are to be used for search, one English and one Swedish. Thereby, it is possible to search in the English text collections, using both English and Swedish search queries. To run it, do the following:

```
cp
↪ /home/dsv/mariask/courses/lis050/lab2/test_text_search.py
↪ ./

python3 test_text_search.py
↪ /home/dsv/mariask/courses/lis050/lab2/search_configuration_1.json
```

You find even more languages in `/home/dsv/robe/lis050/lab2/conceptnet-numberbatch`. If you want, you can copy `search_configuration_1.json` to your own folder, and replace Swedish with some other language(s), but please do not load more than two languages (one of which must be English) since the Concept NumberBatch data is large and loading it requires quite a bit of memory.

Now try to some searches. For instance, the following should return some relevant hits:

- “denmark” (en); “danmark” (sv); “dänemark” (de)
- “jupiter comet” (en); “jupiter komet” (de/sv)
- “automotive industry” (en); “bilindustri” (sv)
- “bless” (en); “välsigna” (sv)

As a preliminary sanity check, you can also verify that the cosine similarity between the following Swedish and English sentence vectors obtained from `WordVectors.make_sentence_vector` should be equal (within rounding errors) to 0.6432: “min hund gillar att äta” and “my dog likes eating”.

4 Pass with distinction: Multilingual word vectors

4.1 Introduction

The *inverted indexing* method of Søgaard et al. (2015) is a simple method for creating multilingual word vectors of the type that was used in Section 3.

Section 2.2 in their paper describes the method. In brief, you create one large matrix for all languages. In this matrix, each column represents a given text (e.g., a Wikipedia article, or in our case, a sentence from the European parliament proceedings) that has been translated into multiple languages, and each row represents a particular word in any of the languages included in the data. For instance, the row that corresponds to the English word *dog* has the value 1 in the rows where the sentence contains ‘dog’, otherwise 0. The row representing the Swedish word ‘hund’ (*dog*) is probably rather similar, since that is likely to occur in the same sentences as ‘dog’.

In contrast, the row representing the Swedish word ‘dog’ (*died*) is likely different. This example illustrates the importance of keeping track of which language a word belongs to. For this you should use the same mechanism as in the ConceptNet Numberbatch word vectors: prepend the language code to each word. For instance, the three words discussed above should be called **en/dog**, **sv/hund** and **sv/dog** respectively.

Include only words that occur at least 10 times in the data, and only include sentences that occur in *all* of the input files.

The matrix is very large, but most elements are zero. This allows more efficient storage, so rather than storing the full matrix as a NumPy array, use a `scipy.sparse.csr_array`. Please refer to its documentation for details. The documentation lists you five ways of creating such a matrix, and number 4 is probably the most convenient in this case. A somewhat simpler (but also somewhat slower) way is to use the `scipy.sparse.dok_array` class instead and fill it in incrementally, similar to the example given in the documentation.

This large matrix should be reduced so that it has 100 columns rather than tens of thousands (i.e., the number of sentences in the data). This can be done using the TruncatedSVD class from scikit-learn, using code like this:

```
svd = TruncatedSVD(n_components=100)
svd.fit(sparse_matrix)
word_vectors = svd.transform(sparse_matrix)
```

After this, `word_vectors` will be an ordinary NumPy array with as many rows as there are words in the vocabulary, and 100 columns.

For compatibility with the ConceptNet Numberbatch vectors, lower-case all words.

4.2 Task

Implement inverted indexing to create word vectors. You can use the text files in the following directory, which contain multi-parallel data from the European parliament:

`/home/dsv/mariask/courses/lis050/lab2/euoparl`

The first part of the filename contains the language code, just as in the word vector files we used in the first part.

Test that the vectors you have produced seem reasonable by checking the cosine similarity between words that you know should be similar. For instance, translations of a word into the different languages should normally be very similar. For instance, what cosine similarities do you get for the word for “Sweden” in different languages? (Also, check the cosine similarity for words that you know should be different.)

Produce the files `de.vec.gz`, `en.vec.gz`, etc. of the same format as ConceptNet Numberbatch. Try them out by using your multilingual search code. Note that the Europarl data belongs to a rather narrow domain, so do not expect to find e.g. “jupiter” in its vocabulary. The “denmark” search (in different languages) should work, as should “parlamentet” (Swedish: “the parliament”).

References

- Jurafsky, D. and Martin, J. H. (2024). *Speech and Language Processing*. 3rd edition (draft), available online at <https://web.stanford.edu/~jurafsky/slp3/>.
- Søgaard, A., Agić, Ž., Martínez Alonso, H., Plank, B., Bohnet, B., and Johannsen, A. (2015). Inverted indexing for cross-lingual NLP. In Zong, C. and Strube, M., editors, *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1713–1722, Beijing, China. Association for Computational Linguistics.